

Part A

1. SQL vs NoSQL

SQL (relational databases) use tables with a fixed structure and clear relationships between data.

NoSQL databases are more flexible and can store data like JSON without a strict structure.

Example:

- SQL → online store (needs relationships and transactions)
- NoSQL → social media or chat app (has a lot of unstructured data)

2. What to choose for the bookstore?

I recommend a **relational database (SQL)**

Because:

- there are relationships (books and authors, orders and customers)
- order data must be accurate
- we need to use JOIN queries

3. ACID Properties

- **Atomicity** — everything happens or nothing happens
- **Consistency** — data stays correct
- **Isolation** — transactions do not affect each other
- **Durability** — data is saved and not lost

4. Primary Key and Foreign Key

- **Primary key** — a unique ID for each row in a table
- **Foreign key** — a column that links to another table

Example:

- Customers(customer_id) → primary key
- Orders.customer_id → foreign key

It connects orders to customers

5. Why avoid data redundancy?

Data redundancy means repeating the same data.
It can cause errors and wastes space.

Normalization:

- splits data into different tables
- removes duplicate data
- makes the database clean and organized

Part B

Authors

author_id (PK)
name
birth_year

Books

isbn (PK)
title
publisher
publication_year
price
stock_quantity

BookAuthors

book_isbn (FK)
author_id (FK)
PRIMARY KEY (book_isbn, author_id)

Customers

customer_id (PK)
name

email (UNIQUE)
address
registration_date

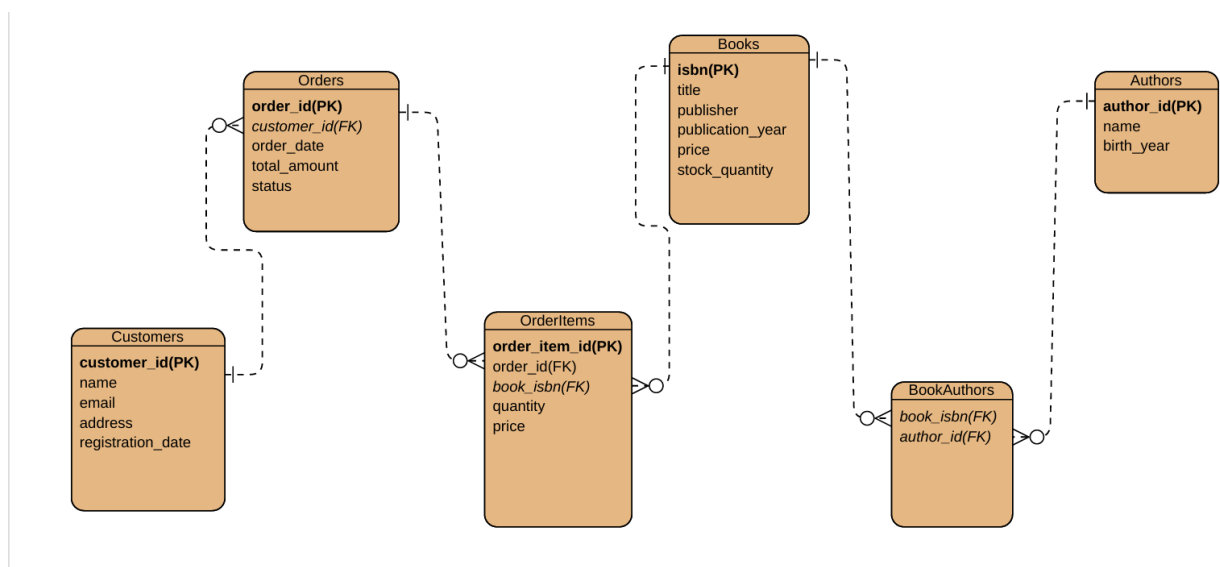
Orders

order_id (PK)
customer_id (FK)
order_date
total_amount
status

OrderItems

order_item_id (PK)
order_id (FK)
book_isbn (FK)
quantity
price

ER - diagram



Partc C DDL

CREATE TABLE Authors (

```
author_id INT PRIMARY KEY,  
name VARCHAR(100) NOT NULL,  
birth_year INT  
);
```

```
CREATE TABLE Books (  
    isbn VARCHAR(20) PRIMARY KEY,  
    title VARCHAR(200) NOT NULL,  
    publisher VARCHAR(100),  
    publication_year INT,  
    price DECIMAL(10,2),  
    stock_quantity INT  
);
```

```
CREATE TABLE BookAuthors (  
    book_isbn VARCHAR(20),  
    author_id INT,  
    PRIMARY KEY (book_isbn, author_id),  
    FOREIGN KEY (book_isbn) REFERENCES Books(isbn)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    FOREIGN KEY (author_id) REFERENCES Authors(author_id)  
        ON DELETE CASCADE ON UPDATE CASCADE  
);
```

```
CREATE TABLE Customers (  
    customer_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100) UNIQUE,  
    address TEXT,  
    registration_date DATE
```

);

CREATE TABLE Orders (

order_id INT PRIMARY KEY,

customer_id INT,

order_date DATE,

total_amount DECIMAL(10,2),

status VARCHAR(20),

FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)

ON DELETE CASCADE ON UPDATE CASCADE

);

CREATE TABLE OrderItems (

order_item_id INT PRIMARY KEY,

order_id INT,

book_isbn VARCHAR(20),

quantity INT,

price DECIMAL(10,2),

FOREIGN KEY (order_id) REFERENCES Orders(order_id)

ON DELETE CASCADE ON UPDATE CASCADE,

FOREIGN KEY (book_isbn) REFERENCES Books(isbn)

ON DELETE CASCADE ON UPDATE CASCADE

);

Part D

INSERT INTO Authors VALUES

(1, 'J.K. Rowling', 1965),

(2, 'George Orwell', 1903),

(3, 'J.R.R. Tolkien', 1892),

(4, 'Agatha Christie', 1890),

(5, 'Dan Brown', 1964);

```
Query  Query History
1  ✓ INSERT INTO Authors VALUES
2    (1, 'J.K. Rowling', 1965),
3    (2, 'George Orwell', 1903),
4    (3, 'J.R.R. Tolkien', 1892),
5    (4, 'Agatha Christie', 1890),
6    (5, 'Dan Brown', 1964);
7
```

INSERT INTO Books VALUES

('B1', 'Harry Potter 1', 'Bloomsbury', 1997, 20.00, 50),
('B2', 'Harry Potter 2', 'Bloomsbury', 1998, 22.00, 40),
('B3', '1984', 'Secker & Warburg', 1949, 15.00, 30),
('B4', 'Animal Farm', 'Secker & Warburg', 1945, 12.00, 25),
('B5', 'The Hobbit', 'Allen & Unwin', 1937, 18.00, 35),
('B6', 'Lord of the Rings', 'Allen & Unwin', 1954, 30.00, 20),
('B7', 'Murder on the Orient Express', 'Collins Crime Club', 1934, 14.00, 15),
('B8', 'The Da Vinci Code', 'Doubleday', 2003, 25.00, 45),
('B9', 'Angels and Demons', 'Pocket Books', 2000, 23.00, 40),
('B10', 'Fantastic Beasts', 'Bloomsbury', 2001, 17.00, 20);

```
✓ INSERT INTO Books VALUES
('B1', 'Harry Potter 1', 'Bloomsbury', 1997, 20.00, 50),
('B2', 'Harry Potter 2', 'Bloomsbury', 1998, 22.00, 40),
('B3', '1984', 'Secker & Warburg', 1949, 15.00, 30),
('B4', 'Animal Farm', 'Secker & Warburg', 1945, 12.00, 25),
('B5', 'The Hobbit', 'Allen & Unwin', 1937, 18.00, 35),
('B6', 'Lord of the Rings', 'Allen & Unwin', 1954, 30.00, 20),
('B7', 'Murder on the Orient Express', 'Collins Crime Club', 1934, 14.00, 15),
('B8', 'The Da Vinci Code', 'Doubleday', 2003, 25.00, 45),
('B9', 'Angels and Demons', 'Pocket Books', 2000, 23.00, 40),
('B10', 'Fantastic Beasts', 'Bloomsbury', 2001, 17.00, 20);
```

INSERT INTO BookAuthors VALUES

```
('B1', 1),  
( 'B2', 1),  
( 'B10', 1),  
( 'B3', 2),  
( 'B4', 2),  
( 'B5', 3),  
( 'B6', 3),  
( 'B7', 4),  
( 'B8', 5),  
( 'B9', 5),  
( 'B6', 5);
```

Query History

```
✓ INSERT INTO BookAuthors VALUES  
  ('B1', 1),  
  ('B2', 1),  
  ('B10', 1),  
  ('B3', 2),  
  ('B4', 2),  
  ('B5', 3),  
  ('B6', 3),  
  ('B7', 4),  
  ('B8', 5),  
  ('B9', 5),  
  ('B6', 5);
```

INSERT INTO Customers VALUES

```
(1, 'Alice', 'alice@mail.com', 'Street 1', '2023-01-10'),  
(2, 'Bob', 'bob@mail.com', 'Street 2', '2023-06-15'),  
(3, 'Charlie', 'charlie@mail.com', 'Street 3', '2022-12-20'),  
(4, 'Diana', 'diana@mail.com', 'Street 4', '2024-01-05'),  
(5, 'Eve', 'eve@mail.com', 'Street 5', '2023-09-09');
```

Query	Query History
1	▼ INSERT INTO Customers VALUES
2	(1, 'Alice', 'alice@mail.com', 'Street 1', '2023-01-10'),
3	(2, 'Bob', 'bob@mail.com', 'Street 2', '2023-06-15'),
4	(3, 'Charlie', 'charlie@mail.com', 'Street 3', '2022-12-20'),
5	(4, 'Diana', 'diana@mail.com', 'Street 4', '2024-01-05'),
6	(5, 'Eve', 'eve@mail.com', 'Street 5', '2023-09-09');
7	

INSERT INTO Orders VALUES

(1, 1, '2024-01-10', 35.00, 'pending'),
(2, 2, '2024-02-05', 50.00, 'shipped'),
(3, 3, '2024-02-20', 18.00, 'pending'),
(4, 1, '2024-03-01', 45.00, 'shipped'),
(5, 4, '2024-03-10', 60.00, 'pending'),
(6, 5, '2024-03-15', 25.00, 'shipped'),
(7, 2, '2024-03-18', 40.00, 'pending'),
(8, 3, '2024-03-20', 55.00, 'shipped');

Query	Query History
1	▼ INSERT INTO Orders VALUES
2	(1, 1, '2024-01-10', 35.00, 'pending'),
3	(2, 2, '2024-02-05', 50.00, 'shipped'),
4	(3, 3, '2024-02-20', 18.00, 'pending'),
5	(4, 1, '2024-03-01', 45.00, 'shipped'),
6	(5, 4, '2024-03-10', 60.00, 'pending'),
7	(6, 5, '2024-03-15', 25.00, 'shipped'),
8	(7, 2, '2024-03-18', 40.00, 'pending'),
9	(8, 3, '2024-03-20', 55.00, 'shipped');
0	

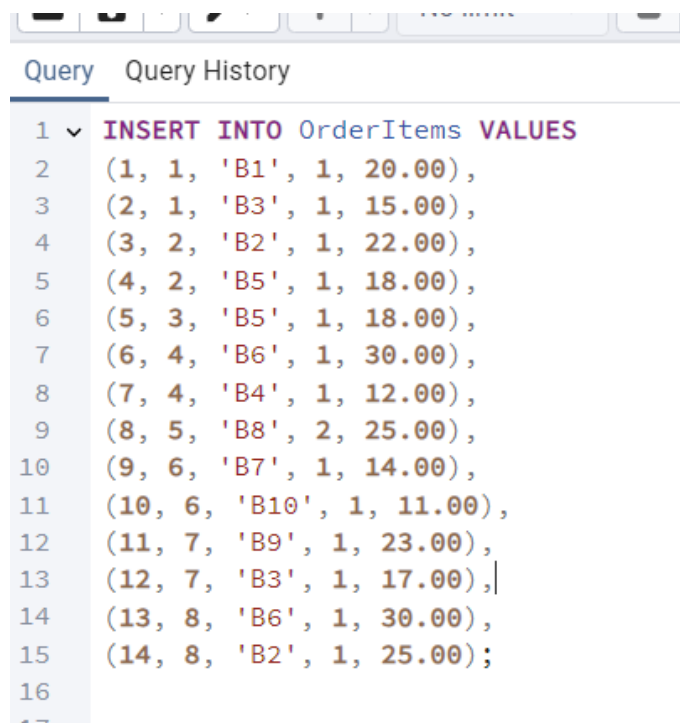
INSERT INTO OrderItems VALUES

(1, 1, 'B1', 1, 20.00),
(2, 1, 'B3', 1, 15.00),

```

(3, 2, 'B2', 1, 22.00),
(4, 2, 'B5', 1, 18.00),
(5, 3, 'B5', 1, 18.00),
(6, 4, 'B6', 1, 30.00),
(7, 4, 'B4', 1, 12.00),
(8, 5, 'B8', 2, 25.00),
(9, 6, 'B7', 1, 14.00),
(10, 6, 'B10', 1, 11.00),
(11, 7, 'B9', 1, 23.00),
(12, 7, 'B3', 1, 17.00),
(13, 8, 'B6', 1, 30.00),
(14, 8, 'B2', 1, 25.00);

```



Part E: SQL Queries

1. List all books with title, authors, and price

```

SELECT b.title, a.name AS author, b.price
FROM Books b
JOIN BookAuthors ba ON b.isbn = ba.book_isbn
JOIN Authors a ON ba.author_id = a.author_id;

```

	title character varying (200)	author character varying (100)	price numeric (10,2)
1	Harry Potter 1	J.K. Rowling	20.00
2	Harry Potter 2	J.K. Rowling	22.00
3	Fantastic Beasts	J.K. Rowling	17.00
4	1984	George Orwell	15.00
5	Animal Farm	George Orwell	12.00
6	The Hobbit	J.R.R. Tolkien	18.00
7	Lord of the Rings	J.R.R. Tolkien	30.00
8	Murder on the Orient Express	Agatha Christie	14.00
9	The Da Vinci Code	Dan Brown	25.00
10	Angels and Demons	Dan Brown	23.00
11	Lord of the Rings	Dan Brown	30.00

2. Customers who registered in 2023

```
SELECT *
FROM Customers
WHERE EXTRACT(YEAR FROM registration_date) = 2023;
```

	customer_id [PK] integer	name character varying (100)	email character varying (100)	address text	registration_date date
1	1	Alice	alice@mail.com	Street 1	2023-01-10
2	2	Bob	bob@mail.com	Street 2	2023-06-15
3	5	Eve	eve@mail.com	Street 5	2023-09-09

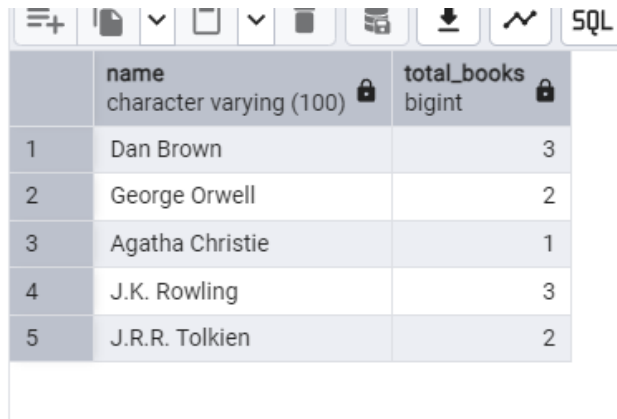
3. Pending orders in last 30 days

```
SELECT *
FROM Orders
WHERE status = 'pending'
AND order_date >= DATE '2024-03-01' - INTERVAL '30 days';
```

	order_id [PK] integer	customer_id integer	order_date date	total_amount numeric (10,2)	status character varying (20)
1	3	3	2024-02-20	18.00	pending
2	5	4	2024-03-10	60.00	pending
3	7	2	2024-03-18	40.00	pending

4. Count books per author

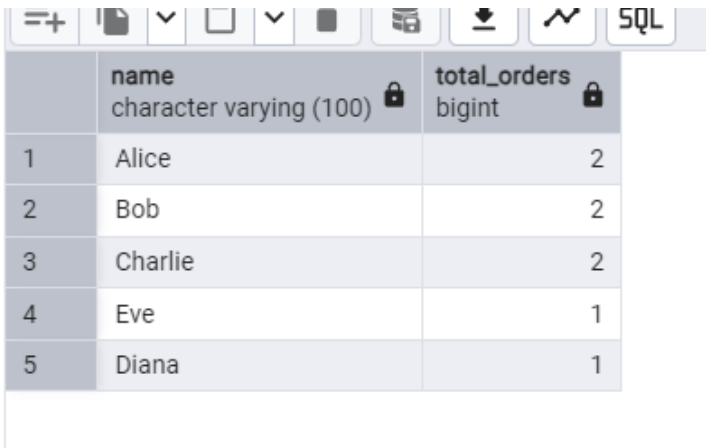
```
SELECT a.name, COUNT(ba.book_isbn) AS total_books
FROM Authors a
JOIN BookAuthors ba ON a.author_id = ba.author_id
GROUP BY a.name;
```



	name character varying (100)	total_books bigint
1	Dan Brown	3
2	George Orwell	2
3	Agatha Christie	1
4	J.K. Rowling	3
5	J.R.R. Tolkien	2

5. Total orders per customer (descending)

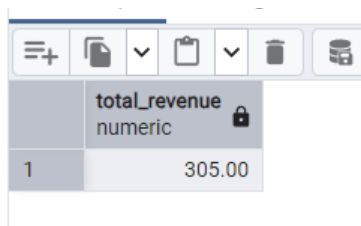
```
SELECT c.name, COUNT(o.order_id) AS total_orders
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.name
ORDER BY total_orders DESC;
```



	name character varying (100)	total_orders bigint
1	Alice	2
2	Bob	2
3	Charlie	2
4	Eve	1
5	Diana	1

6. Total revenue (from OrderItems)

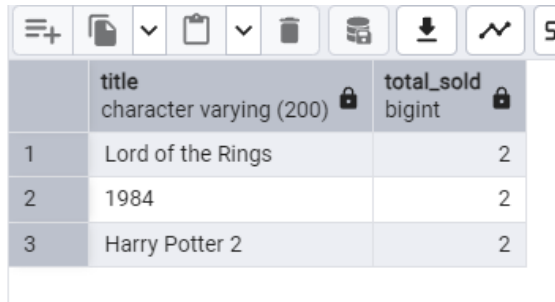
```
SELECT SUM(quantity * price) AS total_revenue
FROM OrderItems;
```



	total_revenue numeric
1	305.00

7. Top 3 best-selling books

```
SELECT b.title, SUM(oi.quantity) AS total_sold
FROM OrderItems oi
JOIN Books b ON oi.book_isbn = b.isbn
GROUP BY b.title
ORDER BY total_sold DESC
LIMIT 3;
```



	title character varying (200)	total_sold bigint
1	Lord of the Rings	2
2	1984	2
3	Harry Potter 2	2

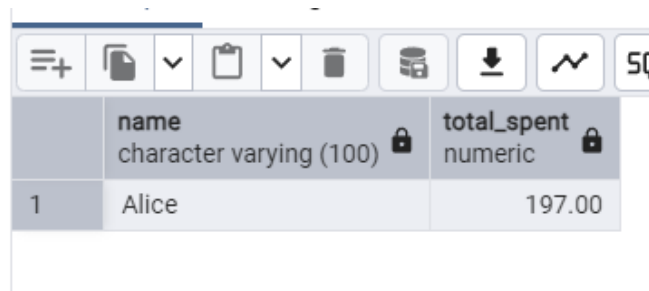
8. Customers who spent more than \$100

```
SELECT c.name, SUM(oi.quantity * oi.price) AS total_spent
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN OrderItems oi ON o.order_id = oi.order_id
GROUP BY c.name
HAVING SUM(oi.quantity * oi.price) > 100;

UPDATE OrderItems

SET quantity = 5

WHERE order_id = 4 AND book_isbn = 'B6';
```



	name character varying (100)	total_spent numeric
1	Alice	197.00

9. Orders and sales per month (2024)

```
SELECT
    EXTRACT(MONTH FROM order_date) AS month,
    COUNT(*) AS total_orders,
    SUM(total_amount) AS total_sales
FROM Orders
WHERE EXTRACT(YEAR FROM order_date) = 2024
GROUP BY month
ORDER BY month;
```

	month numeric 	total_orders bigint 	total_sales numeric 
1	1	1	35.00
2	2	2	68.00
3	3	5	225.00